Set up the environment variable

You can either set up these variables in your *.bashsrc* files or give a series of export commands while building the tool chain.

```
# export target=arm-apple-darwin9    //Sets the target build environment
# export prefix=/toolchain/pre        //Sets the default prefix path
# export sysroot=/toolchain/sys
# export PATH="${prefix}/bin":$PATH
# export cctools=/toolchain/src/cctools
# export gcc=/toolchain/src/gcc
# export csu=/toolchain/src/csu
# export build=/toolchain/bld    //Sets the build directory
```

Once environment variables are set do the following:

1. Install Csu, which provides C hooks into assembly's "start" entry point, and sets up the stack so that your program's main ( ) function can be called.

```
# mkdir -p ${csu}
# cd "${csu}"
# svn co http://iphone-dev.googlecode.com/svn/trunk/csu .
# cp -R *.o "${sysroot}"/usr/lib
# cd "${sysroot}"/usr/lib
# chmod 644 *.o
# cp -Rf crt1.o crt1.10.5.o
# cp -Rf dylib1.o dylib1.10.5.o
```

2. Build the GNU Tool Chain: This includes building a cross-compiling tool chain such as an assembler, linker and so on

```
# rm -rf "${cctools}"
# svn co http://iphone-dev.googlecode.com/svn/branches/odcctools-9.2-ld "${cctools}"
# mkdir -p "${build}"
# cd "${build}"
# mkdir cctools-iphone
# cd cctools-iphone
# CFLAGS=-m32 LDFLAGS=-m32 "${cctools}"/configure --target="${target}" --prefix="${prefix}" --disable-ld64
# make
# make install
```

3. Installing the iPhoneOS Header Files: This is where we use our hard-earned MacOSX10.4u.sdk.

```
# cd "${build}"
# svn co http://iphone-dev.googlecode.com/svn/branches/include-1.2-sdk include
# cd include
# ./configure --prefix="${sysroot}"/usr --with-macosx-sdk=/sdk/MacOSX10.4u.sdk
# bash install-headers.sh
```

4. Building LLVM Compiler (According to Wikipedia: "The Low Level Virtual Machine, generally known as LLVM, is a compiler infrastructure, written in C++, which is designed for compile-time, link-time, run-time, and "idle-time" optimisation of programs written in arbitrary programming languages. For details refer to *en.wikipedia.org/wiki/Low_Level_Virtual_Machine).*

```
# rm -rf "${gcc}"
# git clone git://git.saurik.com/llvm-gcc-4.2 "${gcc}"
# mkdir -p "${build}"
# cd "${build}"
# mkdir gcc-4.2-iphone
# cd gcc-4.2-iphone
# "${gcc}"/configure --target="${target}" \
--prefix="${prefix}" --with-sysroot="${sysroot}" \
--enable-languages=c,c++,objc,obj-c++ \
--with-as="${prefix}"/bin/"${target}"-as \
--with-ld="${prefix}"/bin/"${target}"-ld \
--enable-wchar_t=no \
--with-gxx-include-dir=/usr/include/c++/4.0.0
# make -j2
# make install
# mkdir -p "${sysroot}"/"${dirname "${prefix}"}"
# ln -s "${prefix}" "${sysroot}"/"${dirname "${prefix}"}"
```

After doing all this, you should have binaries listed in Figure 5 in place.

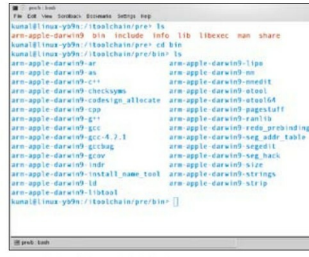

Figure 5: Listing of iPhone tool chain binaries

A HelloWorld application

Let's build and deploy a simple HelloWorld application. To understand this code, you need to learn Objective-C. Apple has extensive documentation on it—check out *developer.apple.com*.

The following is the code for HelloWorldApp.h:

```
#import <CoreFoundation/CoreFoundation.h>
#import <Foundation/Foundation.h>
#import <UIKit/UIWindow.h>
#import <UIKit/UIKit.h>
#import <UIKit/UIApplication.h>
#import <UIKit/UITextView.h>
#import <UIKit/UINavigationBar.h>
#import <UIKit/UIView.h>
#import <UIKit/UIAccelerometer.h>

@interface HelloWorldApp : UIApplication <UIAccelerometerDelegate> {
    UIView          *mainView;
    UITextView      *textView;
```